Luke Colehower
Applied Database Technologies
Lab 2: SELECT statements

Chapter 3: Retrieving data from a single table

1. Write a SELECT statement that returns five columns from the products table: product_id, product_code, product_name, list_price, and discount_percent. Then, run this statement to make sure it works correctly.

   Add an ORDER BY clause to this statement that sorts the result set by product_name in ascending order. Then, run this statement again to make sure it works correctly. This is a good way to build and test a statement, one clause at a time.

   SELECT product_id, product_code, product_name, list_price, discount_percent
   FROM products
   ORDER BY product_name;

2. Write a SELECT statement that returns one column from the customers table named full_name that joins the last_name and first_name columns.

   Using the CONCAT() function, format this column with the last name, a comma, a space, and the first name like this:

   Doe, John

   Sort the result set by the last_name column in descending order.
   Return only the customers whose last name begins with letters from N to Z.

   NOTE: Use ‘<’ or ‘>’ to compare strings of characters.When comparing strings of characters, ‘N’ comes before any string of characters that begins with ‘N’. For example, ‘N’ comes before ‘Nathan’.

   SELECT CONCAT(last_name, ' , ' , first_name) AS 'Full Name'
   FROM customers
   WHERE last_name >'n' AND last_name < 'z'
   ORDER BY last_name DESC ;

3. Write a SELECT statement that returns these columns from the products table:

   - product_name: The product_name column
   - list_price: The list_price column
   - date_added: The date_added column

   Return only the rows with a list price that’s greater than 100 and less than 2000.

Sort the result set by the date_added column in ascending order.

SELECT product_name, list_price, date_added

FROM products

WHERE list_price > 100 AND list_price < 200

ORDER BY date_added;

4. Write a SELECT statement that returns these column names and data from the products table:
    - product_name: The product_name column
    - list_price: The list_price column
    - discount_percent: The discount_percent column
    - discount_amount: A column that's calculated from list_price and discoount_percent
    - discount_price: A column that's calculated from the list_price, discount_percent, and discount_amount.

    Round the discount_amount and discount_price columns to 2 decimal places. Sort the result set by the discount_price column in descending sequence.

    Use the LIMIT clause so the result set contains only the first 5 rows.

    SELECT product_name, list_price, discount_percent,

    ROUND((list_price * discount_percent / 100), 2) AS discount_amount,

    ROUND((list_price - (list_price * discount_percent / 100)), 2) AS discount_price

    FROM products

    ORDER BY discount_price DESC

    LIMIT 5;

5. Write a SELECT statement that returns these column names and data from the Order_Items table:
    - item_id: The item_id column
    - item_price: The item_price column
    - discount_amount: The discount_amount column
    - quantity: The quantity column
    - price_total: A column that's calculated by multiplying the item_price by the quantity
    - discount_total: A column that's calculated by multiplying the discount_amount by the quantity

- item_total: A column that's calculated by subtracting the discount_amount from the item price and then multiplying by the quantity

Only return rows where the item_total is greater than 400.
Sort the result set by the item_total column in descending sequence.

SELECT item_id, item_price, discount_amount, quantity,

item_price * quantity AS price_total,

discount_amount * quantity AS discount_total,

(item_price - discount_amount) * quantity AS item_total

FROM order_items

WHERE

((item_price - discount_amount) * quantity) > 400

ORDER BY item_total DESC;

6. SELECT statement that returns these columns from the Orders table:
   - order_id: The order_id column
   - order_date: The order_date column
   - ship_date: The ship_date column

   Return only the rows where the ship_date column contains a null value.

   SELECT order_id, order_date, ship_date

   FROM orders

   WHERE ship_date IS NULL;

7. Write a SELECT statement without a FROM clause that uses the NOW function to create a row with these columns:
   - today_unformatted: The NOW() function unformatted
   - today_formatted: The NOW() function in the format DD-Mon-YYYY

   This displays a number for the day, an abbreviation for the month, and a four-digit year.

   SELECT NOW() AS today_unformatted, DATE_FORMAT(NOW(), '%d-%b-%Y') AS today_formatted;

8. Write a SELECT statement **without** a FROM clause that creates a row with these columns:
   - 100 (dollars) with a column alias 'price'

- .07 (7 percent) with a column alias ‘tax_rate’
- The price multiplied by the tax_rate with a column alias ‘tax_amount’
- The price plus the tax_amount with a column alias ‘total’

To calculate the third column, add the expressions you used for the first and second columns.

To calculate the fourth column, add the expressions you used for the first and third columns.

SELECT 100 AS price, 0.07 AS tax_rate, (100 * 0.07) AS tax_amount, (100 + (100 * 0.07)) AS total;

Chapter 8: Working with data types

1. Write a SELECT statement that returns these columns from the products table:
   - The list_price column
   - A column that uses the FORMAT() function to return the list_price column with 2 digits to the right of the decimal point
   - A column that uses the CONVERT() function to return the list_price column as an integer
   - A column that uses the CAST() function to return the list_price column as an integer

     SELECT list_price, FORMAT(list_price, 2) AS formatted_price,
     CONVERT(list_price, SIGNED) AS as_integer_convert,
     CAST(list_price AS SIGNED) AS as_integer_Cast
     FROM products;

2. Write a SELECT statement that returns these columns from the products table:
   - The date_added column
   - A column that uses the CAST function to return the date_added column with its date only (year, month, and day)
   - A column that uses the CAST function to return the date_added column with just the year and the month
   - A column that uses the CAST function to return the date_added column with its full time only (hour, minutes, and seconds)

     SELECT date_added, CAST(date_added AS DATE),

     CAST(date_added AS DATE), DATE_FORMAT(DATE(date_added), '%Y-%M') AS date_year_month,

     CAST(date_added AS TIME) AS time_only

FROM products;